

Project 2 Report: Employee Attrition

Stacy Bowler
Utah State University
stacy.bowler@sdl.usu.edu

Derrik Miller
Utah State University
derrik.miller@sdl.usu.edu

2026-08-07

Abstract This project develops a blank.

Introduction

The objective of this project is to develop a predictive model capable of determining which employees are most likely to leave. This can allow the business to focus their retention efforts on high-risk employees.

The business decision addressed by this analysis is:

How much financial value does using this predictive model create compared with not using one?

The resulting model can be used to identify employees that appear likely to leave.

Business Objective

The primary objective is to accurately determine which employees are likely to quit before they do.

A practical utility function for this problem is:

$$U = \begin{cases} 0.60 \times 40000 - 8000 = 16000 & \text{if a retention intervention is provided and prevents attrition} \\ -8000 & \text{if a retention intervention is provided unnecessarily} \\ 0 & \text{if no intervention is provided and the employee leaves} \\ 0 & \text{if no intervention is provided and the employee stays} \end{cases}$$

The utility function reflects the incremental financial impact of acting on the model's recommendations. A true positive represents the expected net savings from successfully retaining an employee, while a false positive represents the cost of an unnecessary retention intervention. False negatives and true negatives are assigned zero utility because they represent the baseline outcome in which no intervention occurs.

assumptions:

- Cost of employee turnover: \$40,000
- Cost of retention intervention: \$8,000
- Retention intervention success rate: 60%

Plan

The analysis follows the regression modeling workflow:

1. Define the business problem.
2. Acquire and clean employee data.
3. Explore relationships between candidate predictors and attrition.
4. Build an interpretable regression model.
5. Evaluate assumptions and model fit.
6. Generate predictions and business recommendations.

The ideal dataset would contain:

- Age
- Attrition
- Salary
- Years at the company
- Overtime
- Job role
- Job satisfaction score

Due to the data being fake generated data, it was more than ideal. It included everything we thought of and much more.

Build

Data Source

The dataset was obtained from Kaggle at <https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset> and contains a fictional dataset created by IBM data scientists.

Variables initially available included 35 different point including notably:

- Age
- Attrition
- Department
- Gender
- Job Role
- Job Satisfaction
- Martial Status
- Monthly Income
- Number of Companies Worked
- Overtime

- Years Since Last Promotion
- Year at Company
- Years in Current Role

Data Cleaning

The following preprocessing steps were performed:

- Converted Attrition to a binary outcome where Yes = 1

```
import os
import numpy as np
import polars as pl
import pandas as pd
import seaborn.objects as so
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf

from patsy import dmatrices
from sklearn.base import clone
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    roc_auc_score
)
from statsmodels.stats.outliers_influence import variance_inflation_factor
# from sklearn.utils import resample

pl.Config.set_tbl_rows(-1)
pl.Config.set_tbl_cols(-1)
pl.Config.set_fmt_str_lengths(100)

employee_data = (pl.read_csv(os.path.join('data', 'HR-Employee-Attrition.csv')))
# Make the outcome binary where 'Yes' = 1
.with_columns(
    pl.when(pl.col('Attrition') == 'Yes').then(1)
    .when(pl.col('Attrition') == 'No').then(0)
```

```

        .alias('Left')
    )
    .select(pl.exclude(['Attrition', 'Department', 'EmployeeNumber', 'DailyRate',
        'MonthlyIncome', 'EmployeeCount', 'Over18', 'PerformanceRating']))
    )

```

Explore

Exploratory Data Analysis

Initial visualizations focused on the relationships between attrition and major numerical predictors.

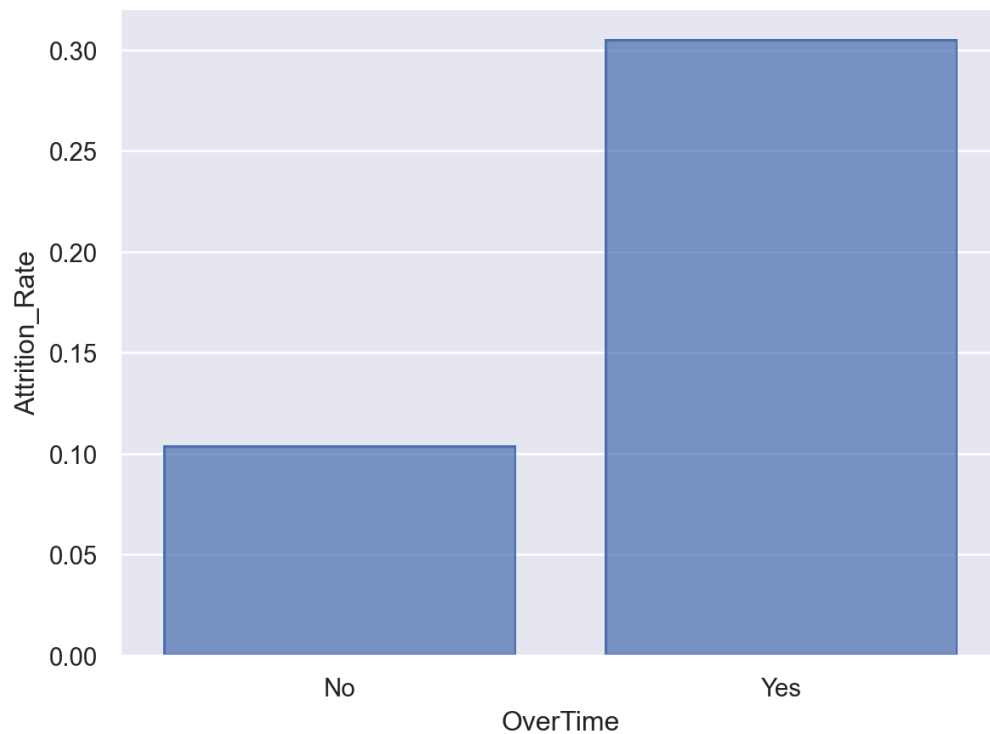
Overtime vs Attrition

```

overtime_rate = (
    employee_data
    .group_by("OverTime")
    .agg(Attrition_Rate=pl.col("Left").mean())
)

so.Plot(overtime_rate.to_pandas(),
        x="OverTime",
        y="Attrition_Rate") \
    .add(so.Bar())

```



This looks like employees that work overtime are more likely to leave.

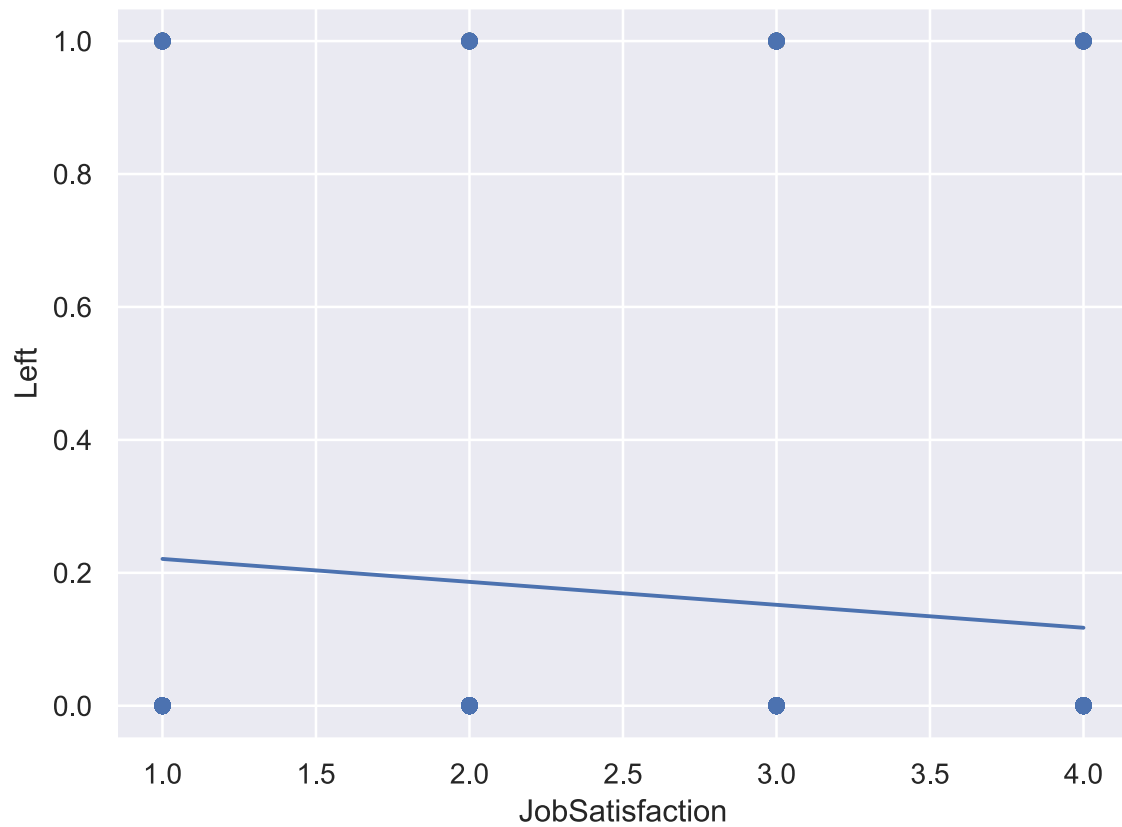
Satisfaction vs Attrition

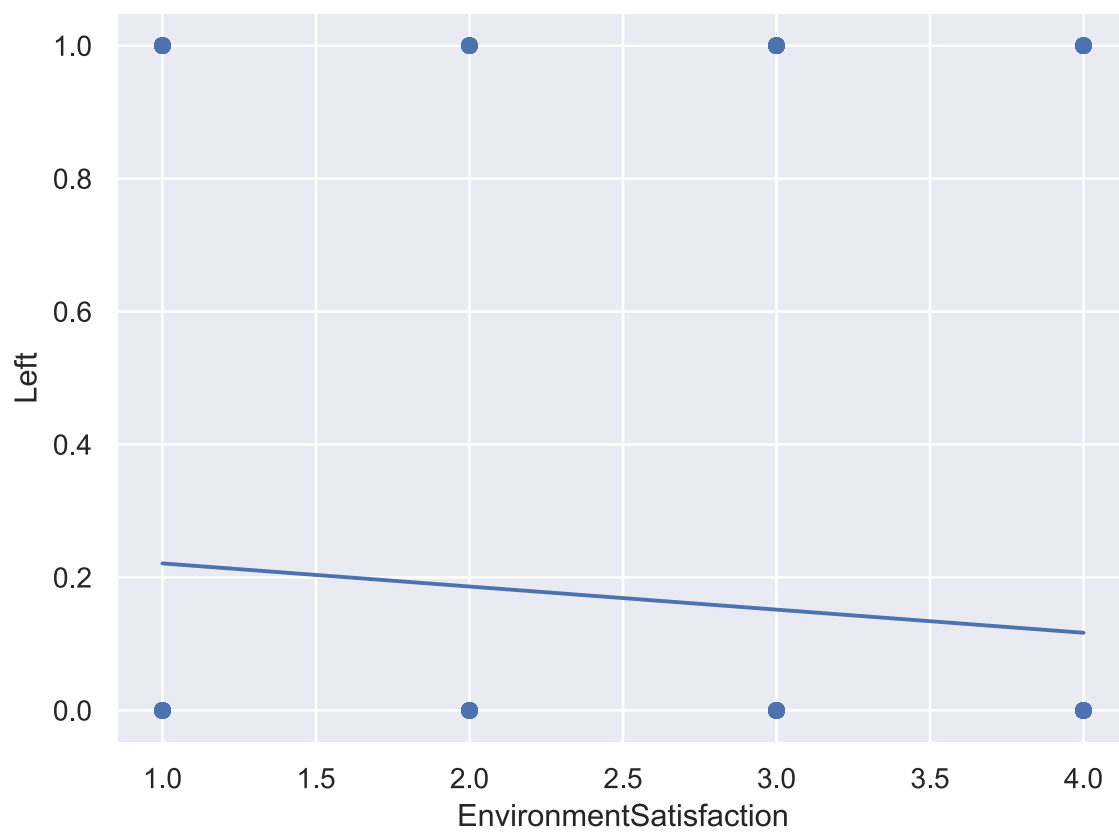
```
(
    so.Plot(employee_data.to_pandas(),
            x="JobSatisfaction",
            y="Left")
    .add(so.Dot(alpha=.20))
    .add(so.Line(), so.PolyFit(order=1))
).show()

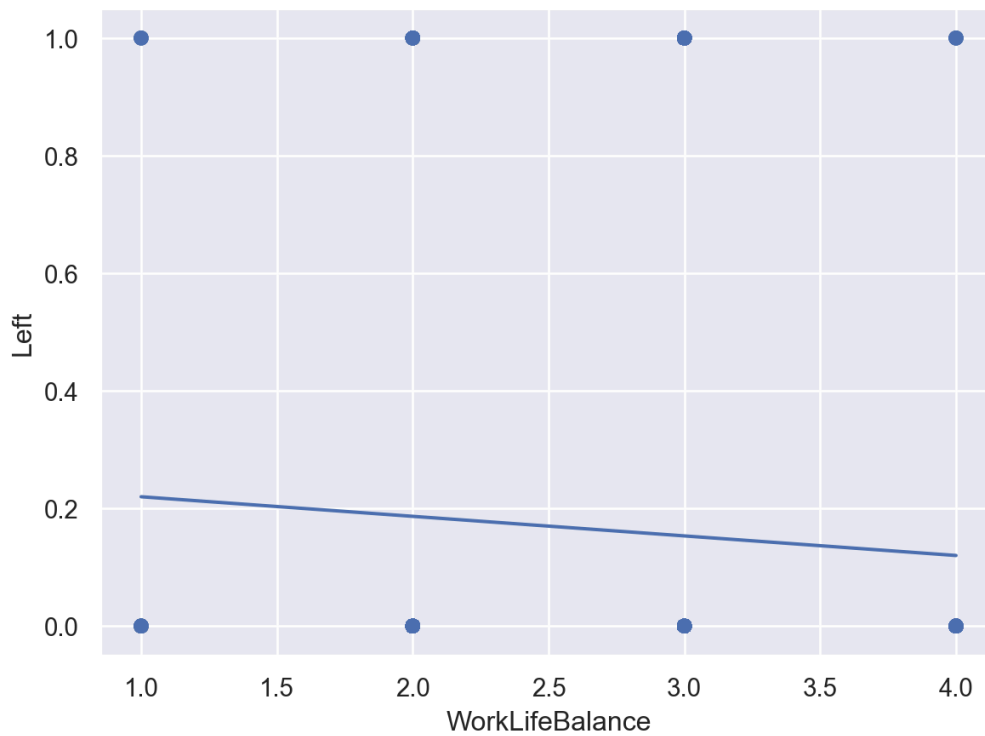
(
    so.Plot(employee_data.to_pandas(),
            x="EnvironmentSatisfaction",
            y="Left")
    .add(so.Dot(alpha=.20))
    .add(so.Line(), so.PolyFit(order=1))
).show()

(
    so.Plot(employee_data.to_pandas(),
            x="WorkLifeBalance",
```

```
y="Left")  
.add(so.Dot(alpha=.20))  
.add(so.Line(), so.PolyFit(order=1))  
)
```



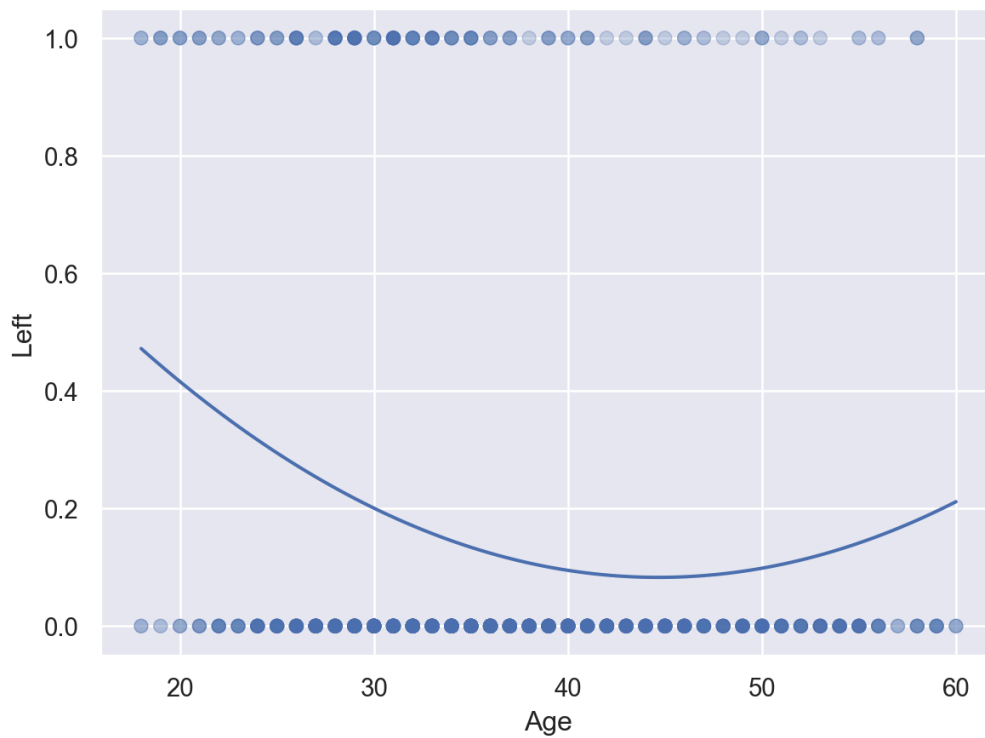




These graphs show what we would expect, that as employees report being more satisfied or having better work/life balance, they are less likely to leave.

Age vs Attrition

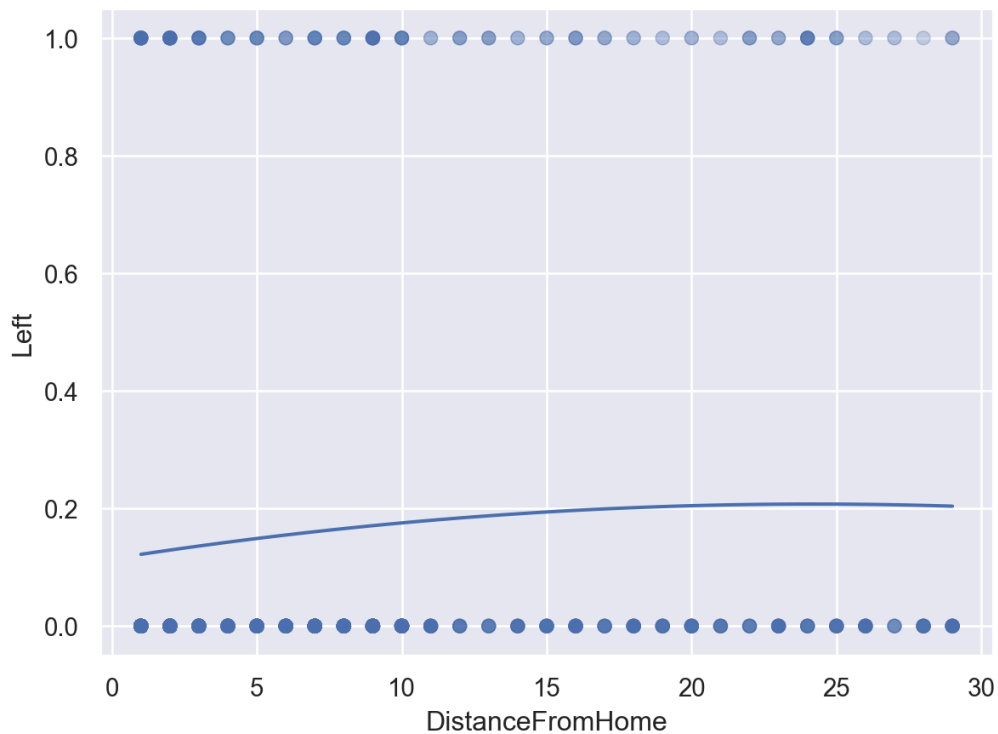
```
(  
    so.Plot(employee_data.to_pandas(),  
            x="Age",  
            y="Left")  
    .add(so.Dot(alpha=.15))  
    .add(so.Line(), so.PolyFit(order=2))  
)
```

We can see that younger employees are generally more likely to leave, but this starts to reverse around age 45. Retirement is likely a factor, but also employees are likely changing jobs for more senior roles.

Commute vs Attrition

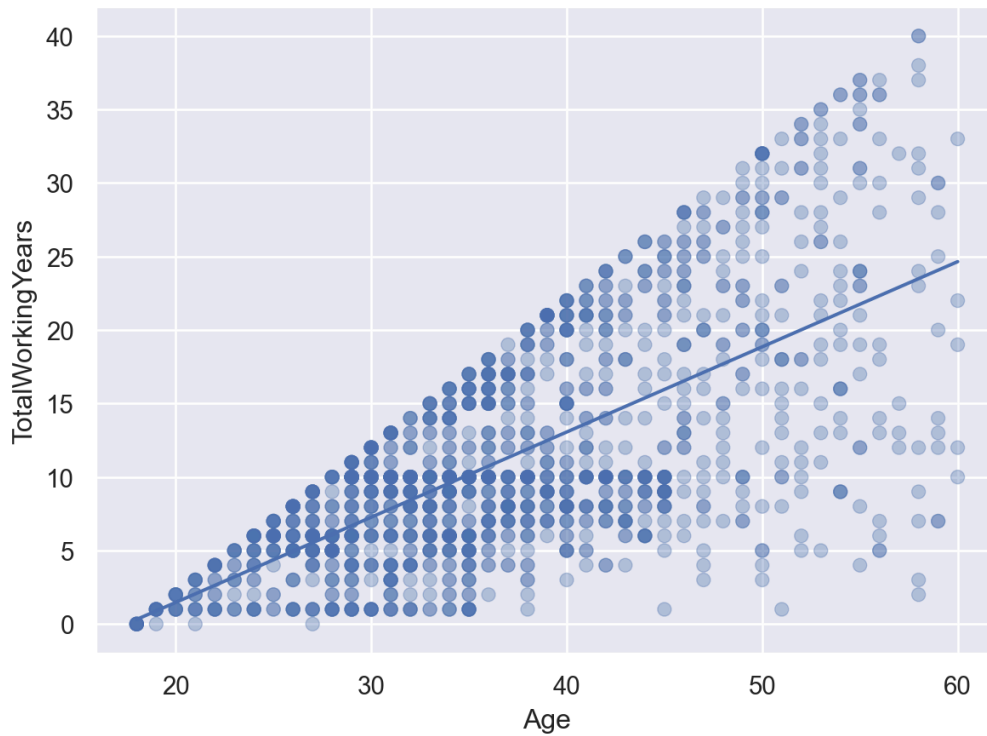
```
(  
    so.Plot(employee_data.to_pandas(),  
            x="DistanceFromHome",  
            y="Left")  
    .add(so.Dot(alpha=.15))  
    .add(so.Line(), so.PolyFit(order=2))  
)
```



There is a fairly weak correlation between how far someone lives from work and if they are likely to leave. It is interesting that the trend appears to level off so it seems like it's only a factor within a certain range.

Age vs Total Working Years

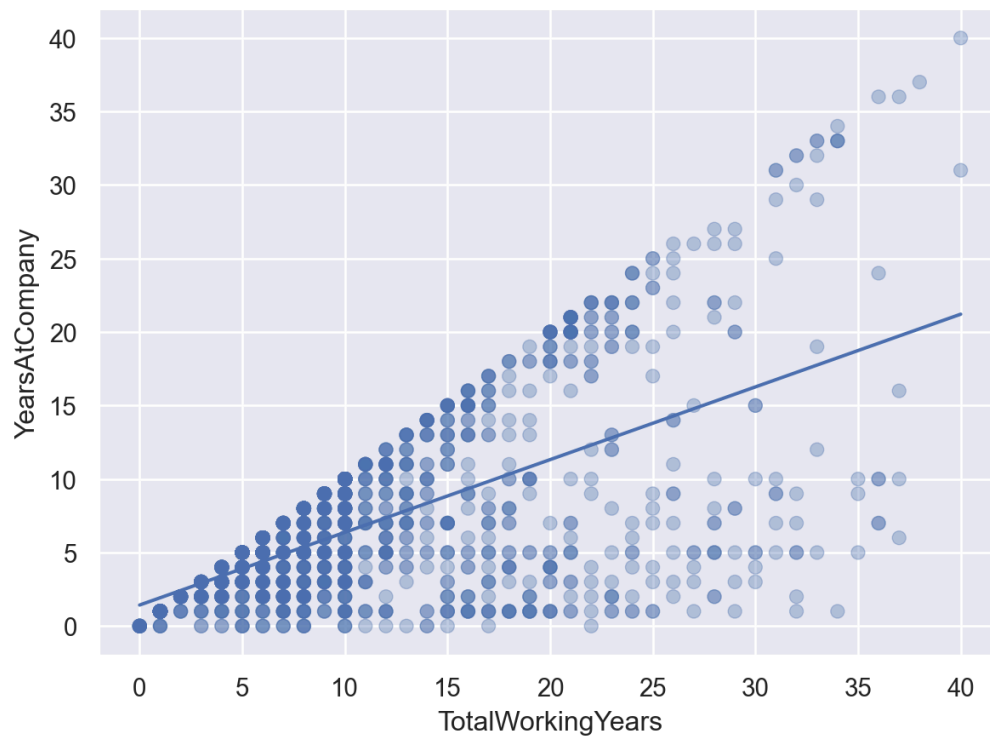
```
(
    so.Plot(employee_data.to_pandas(),
            x="Age",
            y="TotalWorkingYears")
    .add(so.Dot(alpha=.35))
    .add(so.Line(), so.PolyFit(order=1))
)
```



We thought there would be a strong relationship in the data that could cause issues between employee age and total working years. It looks like there is a relationship, but the correlation is not nearly as strong as we thought.

Years at Company vs Total Working Years

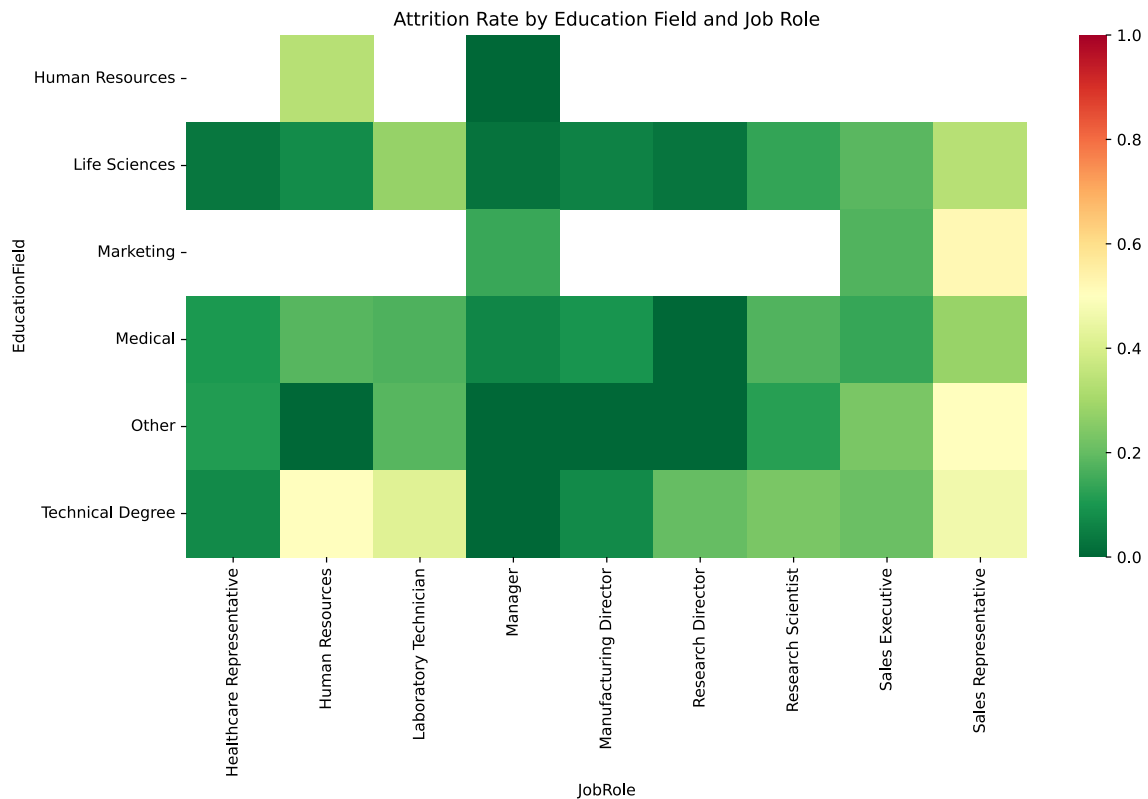
```
(  
    so.Plot(employee_data.to_pandas(),  
            x="TotalWorkingYears",  
            y="YearsAtCompany")  
    .add(so.Dot(alpha=.35))  
    .add(so.Line(), so.PolyFit(order=1))  
)
```



It looks like there is a relationship between years at the company and total working years, but not a very strong one.

Education Field vs Job Role

```
rate = (
    employee_data
    .group_by(["EducationField", "JobRole"])
    .agg(attrition_rate=pl.col("Left").mean())
    .to_pandas()
)
pivot_rate = rate.pivot(
    index="EducationField",
    columns="JobRole",
    values="attrition_rate"
)
plt.figure(figsize=(12,6))
sns.heatmap(pivot_rate, cmap="RdYlGn_r", vmin=0, vmax=1)
plt.title("Attrition Rate by Education Field and Job Role")
plt.show()
```

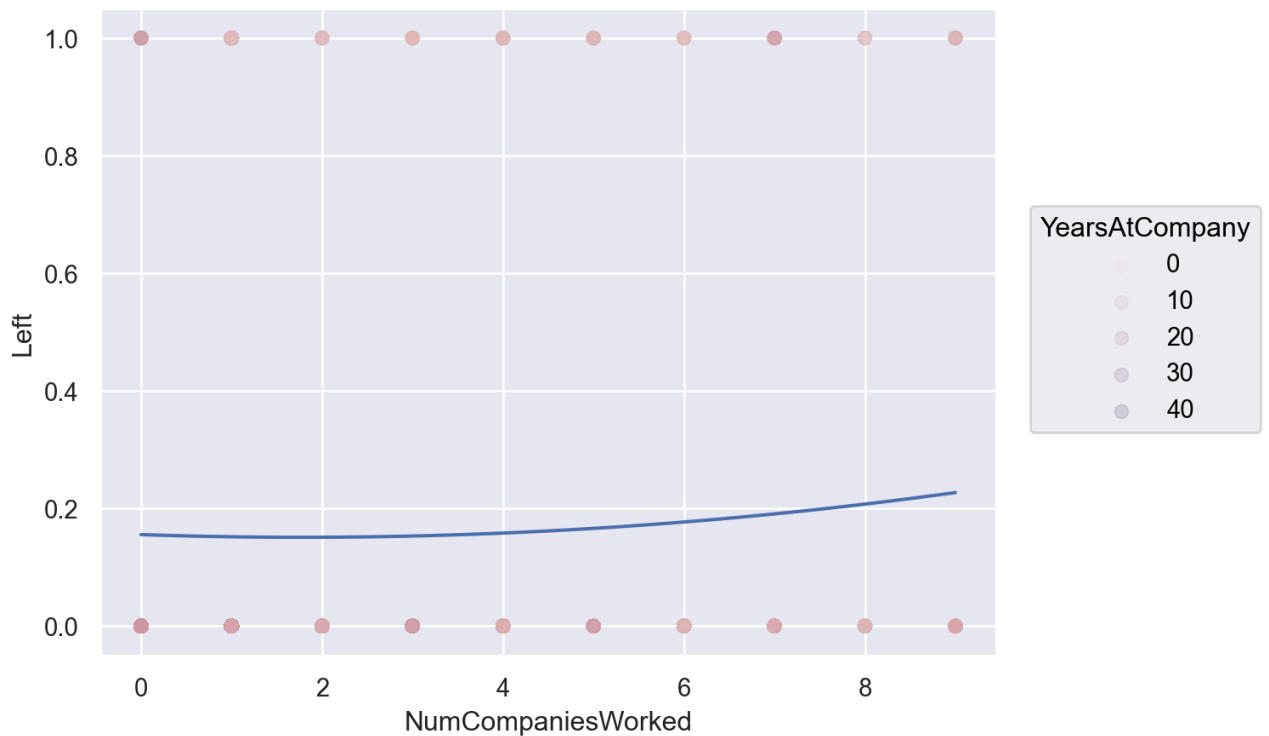


The heatmap reveals a strong structural relationship between education field and job role, indicating that employees are not randomly distributed across roles. Instead, specific education backgrounds concentrate within particular job families. This dependency explains multicollinearity we later observed in the initial logistic regression model.

Attrition varies more meaningfully across job roles than across education fields, suggesting that occupational role is a stronger predictor of turnover than educational background.

Number of Companies and Attrition

```
(
  so.Plot(employee_data.to_pandas(),
    x="NumCompaniesWorked",
    y="Left")
  .add(so.Dot(alpha=.15), color="YearsAtCompany")
  .add(so.Line(), so.PolyFit(order=2))
)
```



This is an interesting graph because it shows that employees that have worked at more companies are more likely to leave. This could be explained by the fact that employees that have worked at more companies are more likely to be looking for new opportunities. It also shows that employees that have been at the company for a long time are less likely to leave, which makes sense.

Key Findings

- Found this.
- Found that.

Reconcile

Sampling Strategy

A stratified sampling approach was used to evenly split test and training data with even attrition and non-attrition employees.

Assumptions

Model Refinements

Several iterations of the model were evaluated.

Adjustments included:

- abcd
- GDs.

- Evaluating Variance Inflation Factors (VIFs).

Hyper-parameter Tuning

```

employee_data = employee_data.to_dummies(
    columns=[
        'BusinessTravel',
        'EducationField',
        'Gender',
        'JobRole',
        'MaritalStatus',
        'OverTime'
    ]
).to_pandas()

# This is a utility function meant to maximize the use of money to retain employees
# It is based on our assumptions:
# * Cost of employee turnover: $40,000
# * Cost of retention intervention: $8,000
# * Retention intervention success rate: 60%
def utility(matrix):
    true_negative, false_positive, false_negative, true_positive = matrix.ravel()

    return (
        true_positive * 16000 +
        false_positive * (-8000) +
        false_negative * 0 +
        true_negative * 0
    )

X = employee_data.drop(columns="Left")
y = employee_data["Left"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Candidate cutoff probabilities and c values
c_values = np.logspace(-2, 2, 15)
thresholds = np.arange(0.20, 0.61, 0.05)

```

```

models = {
    "Ridge": LogisticRegression(l1_ratio=0),
    "Lasso": LogisticRegression(
        l1_ratio=1,
        solver="saga",
        max_iter=5000,
        random_state=42
    ),
    "Elastic": LogisticRegression(
        solver="saga",
        l1_ratio=0.5,
        max_iter=5000,
    )
}

# Setup proper 5-fold cross-validation object
kFolds = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

results = []

# We will try all 3 different models to see which one seems the best
for model_name, base_model in models.items():

    # We are going to tune our cutoff probability and c value at the same time
    best = {
        "C": None,
        "Threshold": None,
        "Utility": -np.inf,
        "Accuracy": 0,
        "Precision": 0,
        "Recall": 0,
        "AUC": 0
    }

    # Loop through each regularization parameter (C)
    for C in c_values:
        for threshold in thresholds:
            fold_utilities = []
            fold_accuracy = []
            fold_precision = []
            fold_recall = []
            fold_auc = []

```



```

# Iterate through the 5 distinct validation folds
for train_idx, val_idx in kFolds.split(X_train, y_train):

    # Split using the index arrays generated by StratifiedKFold
    X_train_fold = X_train.iloc[train_idx]
    X_val_fold = X_train.iloc[val_idx]

    y_train_fold = y_train.iloc[train_idx]
    y_val_fold = y_train.iloc[val_idx]

    # grab our frequentist logistic regression model and set C
    model = Pipeline([
        ("scaler", StandardScaler()),
        ("logistic", clone(base_model).set_params(C=C))
    ])

    model.fit(X_train_fold, y_train_fold)

    probabilities = model.predict_proba(X_val_fold)[:,1]

    predictions = (probabilities >= threshold).astype(int)

    cm = confusion_matrix(y_val_fold, predictions)

    fold_utilities.append(utility(cm))
    fold_accuracy.append(accuracy_score(y_val_fold, predictions))
    fold_precision.append(
        precision_score(y_val_fold, predictions, zero_division=0)
    )
    fold_recall.append(
        recall_score(y_val_fold, predictions, zero_division=0)
    )

    # AUC should use probabilities, not binary predictions
    fold_auc.append(
        roc_auc_score(y_val_fold, probabilities)
    )

avg_utility = np.mean(fold_utilities)
avg_accuracy = np.mean(fold_accuracy)
avg_precision = np.mean(fold_precision)
avg_recall = np.mean(fold_recall)

```

```

avg_auc = np.mean(fold_auc)

if avg_utility > best["Utility"]:
    best["C"] = C
    best["Threshold"] = threshold
    best["Utility"] = avg_utility
    best["Accuracy"] = avg_accuracy
    best["Precision"] = avg_precision
    best["Recall"] = avg_recall
    best["AUC"] = avg_auc

# It's very possible we tie, if so go by accuracy
elif (
    avg_utility == best["Utility"]
    and avg_accuracy > best["Accuracy"]
):
    best["C"] = C
    best["Threshold"] = threshold
    best["Utility"] = avg_utility
    best["Accuracy"] = avg_accuracy
    best["Precision"] = avg_precision
    best["Recall"] = avg_recall
    best["AUC"] = avg_auc

# Save the best settings for this model
results.append({
    "Model": model_name,
    **best
})

# Identify the best cutoff probability, best C, and best model
results = pd.DataFrame(results)

results.sort_values(
    "Utility",
    ascending=False
).head()

```

	Model	C	Threshold	Utility	Accuracy	Precision	Recall	AUC
0	Ridge	0.071969	0.35	260800.0	0.885204	0.671458	0.568421	0.842234
2	Elastic	0.517947	0.40	260800.0	0.889452	0.701604	0.542105	0.841832
1	Lasso	1.000000	0.40	257600.0	0.887753	0.690919	0.542105	0.841591

Elastic Net achieved the highest cross-validated business utility (tied with Ridge), the highest overall accuracy, and the highest precision while maintaining an AUC comparable to the other models. Although Ridge produced slightly higher recall, the difference was small. Therefore, Elastic Net was selected as the final model because it provided the best overall balance of predictive performance and business value.

The similarity in performance suggests that the employee attrition dataset contains a relatively stable set of predictors with limited multicollinearity. Consequently, the choice of regularization method had only a modest impact on predictive performance. Our removal from the data of some predictors where we saw or predicted multicollinearity may have also contributed.

```
final_c = 0.517947
final_probability = 0.40

final_model = Pipeline([
    ("scaler", StandardScaler()),
    ("classification", LogisticRegression(
        solver="saga",
        l1_ratio=0.5,
        C=final_c,
        max_iter=5000,
        random_state=42
    ))
])

final_model.fit(X_train, y_train)

p_test = final_model.predict_proba(X_test)[: , 1]
y_pred = (p_test >= final_probability).astype(int)
final_accuracy = accuracy_score(y_test, y_pred)

final_accuracy

cm = confusion_matrix(y_test, y_pred)
print(utility(cm))

employee_data = employee_data.rename(columns={
    "BusinessTravel_Non-Travel": "BusinessTravel_NonTravel",
    "EducationField_Human Resources": "EducationField_HumanResources",
    "EducationField_Life Sciences": "EducationField_LifeSciences",
    "EducationField_Technical Degree": "EducationField_TechnicalDegree",
    "JobRole_Healthcare Representative": "JobRole_HealthcareRepresentative",
    "JobRole_Human Resources": "JobRole_HumanResources",
    "JobRole_Laboratory Technician": "JobRole_LaboratoryTechnician",
```

```

    "JobRole_Manufacturing Director": "JobRole_ManufacturingDirector",
    "JobRole_Research Director": "JobRole_ResearchDirector",
    "JobRole_Research Scientist": "JobRole_ResearchScientist",
    "JobRole_Sales Executive": "JobRole_SalesExecutive",
    "JobRole_Sales Representative": "JobRole_SalesRepresentative",
  })

predictors = [
  "Age",
  "BusinessTravel_NonTravel",
  "BusinessTravel_Travel_Frequently",
  "BusinessTravel_Travel_Rarely",
  # "C(Department)", # job role perfectly maps to certain departments except
manager
  "DistanceFromHome",
  "EducationField_HumanResources", "EducationField_LifeSciences",
  "EducationField_Marketing", "EducationField_Medical", "EducationField_Other",
  "EducationField_TechnicalDegree", # This seems to be redundant.
  "Education",
  "EnvironmentSatisfaction",
  "Gender_Female",
  "HourlyRate", # Hourly rate seems to group employees together better than other
similar rate columns that tend to only match one or few employees.
  "JobInvolvement",
  "JobLevel",
  "JobRole_HealthcareRepresentative", "JobRole_HumanResources",
  "JobRole_LaboratoryTechnician",
  "JobRole_Manager", "JobRole_ManufacturingDirector", "JobRole_ResearchDirector",
  "JobRole_ResearchScientist",
  "JobRole_SalesExecutive", "JobRole_SalesRepresentative",
  "JobSatisfaction",
  "MaritalStatus_Divorced", "MaritalStatus_Married", "MaritalStatus_Single",
  "MonthlyRate", # may be more linear and likely redundant with hourly rate
  "NumCompaniesWorked",
  "OverTime_Yes",
  "PercentSalaryHike",
  "RelationshipSatisfaction",
  "TotalWorkingYears", # may correlate strongly with Age
  "TrainingTimesLastYear",
  "WorkLifeBalance",
  "YearsAtCompany",
  "YearsInCurrentRole",
  "YearsSinceLastPromotion",

```

```

    "YearsWithCurrManager"
]

formula = "Left ~ " + " + ".join(predictors)

# fit_01 = smf.logit(
#     formula,
#     data=X_train
# ).fit()

# print(fit_01.summary())

# # Create the design matrix used by the model
# y, X = dmatrices(formula, data=pb_train, return_type="dataframe")

# # Calculate VIF
# vif = pd.DataFrame({
#     "Variable": X.columns,
#     "VIF": [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
# })

# print(vif.sort_values("VIF", ascending=False))

```

232000

Given the high P values, we should consider removing these from our model

Variable	p-value
Education 2–5	0.60–0.94
HourlyRate	0.656
PercentSalaryHike	0.584
YearsAtCompany	0.704

Then these next

Variable	p-value
JobLevel 3	0.936
JobLevel 4	0.528
JobLevel 5	0.466
JobRole Manager	0.846

Variable	p-value
JobRole Research Scientist	0.834
Manufacturing Director	0.441

Fit

Final Model

The final model included:

- Param 1
- Param 2

```
# fit_log.summary()
```

The fitted model produced:

- $R^2 = 0.818$
- Adjusted $R^2 = 0.812$
- F-statistic = 141.6
- $p < 0.001$

indicating strong overall explanatory power.

Important Continuous Predictors

Variable	Coefficient
OverTime	2.07
Business Travel	2.02/1.05
Marital Status	1.60

OverTime

Employees working overtime have roughly 8 times the odds of attrition compared with employees who do not, holding the other variables constant:

$$e^{2.07} \approx 7.95$$

Business Travel

Employees traveling frequently have roughly 7.5 times the odds of attrition compared to employees who do not travel, holding the other variables constant:

$$e^{2.02} \approx 7.5$$

Employees traveling rarely also have roughly 2.9 times the odds of attrition compared to employees who do not travel, holding the other variables constant:

$$e^{1.05} \approx 2.9$$

Marital Status

Single employees have roughly 5 times the odds of attrition compared to divorced employees:

$$e^{1.60} \approx 5$$

Evaluate

Residual Analysis

Residual diagnostics indicated:

- No severe violations of linearity.
- Acceptable residual symmetry.
- Some heavy tails remain.
- No extreme multicollinearity concerns.

The largest VIF values were associated with popular truck categories but remained below commonly cited thresholds requiring corrective action.

Limitations

Several limitations should be noted:

- No MSRP information was available.
- Trim level information was not incorporated.
- Accident history was unavailable.
- Maintenance records were unavailable.
- Auction data may not represent all vehicle markets.

These limitations likely explain a portion of the unexplained variance.

Predict

Penalized Regression

```
import numpy as np
import pandas as pd
from patsy import dmatrices
from sklearn.linear_model import LogisticRegressionCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# 1. Prepare Design Matrices using the formula
y, X = dmatrices(formula, data=employee_data, return_type="dataframe")
# Flatten y to a 1D array for scikit-learn
y = y.values.ravel()
```

```

# Split into train/test using the same indices or exact split structure
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

# 2. Scale features (Crucial for regularized models)
# Note: Intercept column from patsy should not be scaled, or can be dropped
# since LogisticRegression adds its own intercept.
X_train_clean = X_train.drop(columns=['Intercept'], errors='ignore')
X_test_clean = X_test.drop(columns=['Intercept'], errors='ignore')

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_clean)
X_test_scaled = scaler.transform(X_test_clean)

# 3. Fit Models with Cross-Validation for Hyperparameter Tuning
cv_folds = 5

# Ridge Regression (L2)
ridge_cv = LogisticRegressionCV(l1_ratios=(0,), Cs=10, cv=cv_folds,
    scoring='neg_log_loss', use_legacy_attributes=True, random_state=42,
    max_iter=1000)
ridge_cv.fit(X_train_scaled, y_train)

# LASSO Regression (L1)
lasso_cv = LogisticRegressionCV(l1_ratios=(1,), solver='saga',
    Cs=10, cv=cv_folds, scoring='neg_log_loss', use_legacy_attributes=True,
    random_state=42, max_iter=5000)
lasso_cv.fit(X_train_scaled, y_train)

# Elastic Net Regression
elastic_cv = LogisticRegressionCV(l1_ratios=[0.1, 0.5, 0.7, 0.9], solver='saga',
    Cs=10, cv=cv_folds, scoring='neg_log_loss', use_legacy_attributes=True,
    random_state=42, max_iter=5000)
elastic_cv.fit(X_train_scaled, y_train)

```

	Cs Cs: int or list of floats, default=10 Each of the values in Cs describes the inverse of regularization strength. If Cs is as an int,	10
--	---	----

	then a grid of Cs values are chosen in a logarithmic scale between 1e-4 and 1e4. Like in support vector machines, smaller values specify stronger regularization.	
	l1_ratios l1_ratios: array-like of shape (n_l1_ratios), default=None Floats between 0 and 1 passed as Elastic-Net mixing parameter (scaling between L1 and L2 penalties). For <code>`l1_ratio = 0`</code> the penalty is an L2 penalty. For <code>`l1_ratio = 1`</code> it is an L1 penalty. For <code>`0 < l1_ratio < 1`</code>, the penalty is a combination of L1 and L2. All the values of the given array-like are tested by cross-validation and the one giving the best prediction score is used. .. warning:: Certain values of <code>`l1_ratios`</code>, i.e. some penalties, may not work with some solvers. See the parameter <code>`solver`</code> below, to know the compatibility between the penalty and solver. .. deprecated:: 1.8 <code>`l1_ratios=None`</code> is deprecated in 1.8 and will raise an error	[0.1, 0.5, ...]

	in version 1.10. Default value will change from <code>'None'</code> to <code>'(0.0,)'</code> in version 1.10.	
	<code>fit_intercept</code> <code>fit_intercept</code>: bool, default=True Specifies if a constant (a.k.a. bias or intercept) should be added to the decision function.	True
	cv <code>cv</code>: int or cross-validation generator, default=None The default cross-validation generator used is Stratified K-Folds. If an integer is provided, it specifies the number of folds, <code>'n_folds'</code>, used. See the module <code>:mod:`sklearn.model_selection`</code> module for the list of possible cross-validation objects. .. versionchanged:: 0.22 <code>'cv'</code> default value if None changed from 3-fold to 5-fold.	5
	<code>dual</code> <code>dual</code>: bool, default=False Dual (constrained) or primal (regularized, see also <code>:ref:`this equation`</code>) formulation. Dual formulation is only implemented for l2 penalty with liblinear solver. Prefer <code>dual=False</code> when <code>n_samples > n_features</code>.	False
	<code>penalty</code> <code>penalty</code>: {'l1', 'l2', 'elasticnet'}, default='l2'	'deprecated'

	Specify the norm of the penalty: - <code>'l2'</code>: add a L2 penalty term (used by default); - <code>'l1'</code>: add a L1 penalty term; - <code>'elasticnet'</code>: both L1 and L2 penalty terms are added. .. warning:: Some penalties may not work with some solvers. See the parameter <code>'solver'</code> below, to know the compatibility between the penalty and solver. .. deprecated:: 1.8 <code>'penalty'</code> was deprecated in version 1.8 and will be removed in 1.10. Use <code>'l1_ratio'</code> instead. <code>'l1_ratio=0'</code> for <code>'penalty='l2'</code>, <code>'l1_ratio=1'</code> for <code>'penalty='l1'</code> and <code>'l1_ratio'</code> set to any float between 0 and 1 for <code>'penalty='elasticnet'</code>.	
	scoring scoring: str or callable, default=None The scoring method to use for cross-validation. Options: - str: see :ref:`scoring_string_names` for options. - callable: a scorer callable object (e.g., function) with sig-	'neg_log_loss'

	nature <pre>``scorer(estimator, X, y)``.</pre> See :ref:`scoring_callable` for details. - `None`: :ref:`accuracy` is used.	
	solver solver: {'lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky', 'sag', 'saga'}, default='lbfgs' Algorithm to use in the optimization problem. Default is 'lbfgs'. To choose a solver, you might want to consider the following aspects: - 'lbfgs' is a good default solver because it works reasonably well for a wide class of problems. - For :term:`multiclass` problems (<code>n_classes >= 3</code>), all solvers except 'liblinear' minimize the full multinomial loss, 'liblinear' will raise an error. - 'newton-cholesky' is a good choice for <code>n_samples >> n_features * n_classes</code>, especially with one-hot encoded categorical features with rare categories. Be aware that the memory usage of this solver has a quadratic dependency on <code>n_features * n_classes</code> because it explicitly com-	'saga'

puts the full Hessian matrix.
 - For small datasets, 'liblinear' is a good choice, whereas 'sag' and 'saga' are faster for large ones;
 - 'liblinear' might be slower in :class:`LogisticRegressionCV` because it does not handle warm-starting.
 - 'liblinear' can only handle binary classification by default. To apply a one-versus-rest scheme for the multiclass setting one can wrap it with the :class:`~sklearn.multiclass.OneVsRestClassifier`.

.. warning::
 The choice of the algorithm depends on the penalty ('l1_ratio=0` for L2-penalty, 'l1\_ratio=1` for L1-penalty and `0 < l1\_ratio < 1` for Elastic-Net) chosen and on (multinomial) multiclass support:

```

=====
=====
solver l1_ratio multinomial
multiclass
=====
=====
'lbfgs' l1_ratio=0 yes
'liblinear' l1_ratio=1 or
l1_ratio=0 no
'newton-cg' l1_ratio=0 yes
'newton-cholesky' l1_ratio=0
yes
'sag' l1_ratio=0 yes

```

	'saga' 0<=l1_ratio<=1 yes =====	
	.. note:: 'sag' and 'saga' fast convergence is only guaranteed on features with approximately the same scale. You can preprocess the data with a scaler from :mod:`sklearn.preprocessing`. .. versionadded:: 0.17 Stochastic Average Gradient (SAG) descent solver. Multinomial support in version 0.18. .. versionadded:: 0.19 SAGA solver. .. versionadded:: 1.2 newton-cholesky solver. Multinomial support in version 1.6.	
	tol tol: float, default=1e-4 Tolerance for stopping criteria.	0.0001
	max_iter max_iter: int, default=100 Maximum number of iterations of the optimization algorithm.	5000
	class_weight class_weight: dict or 'balanced', default=None Weights associated with classes in the form ``{class_label: weight}``.	None

	If not given, all classes are supposed to have weight one. The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as <code>n_samples / (n_classes * np.bincount(y))</code>. Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified. .. versionadded:: 0.17 class_weight == 'balanced'	
	<code>n_jobs</code> <code>n_jobs</code>: int, default=None Number of CPU cores used during the cross-validation loop. <code>None</code> means 1 unless in a <code>:obj: joblib.parallel_backend` context.</code></p> <p><code>-1</code> means using all processors. See <code>:term: Glossary` for more details.</code>	None
	<code>verbose</code> <code>verbose</code>: int, default=0 For the 'liblinear', 'sag' and 'lbfgs' solvers set verbose to any positive number for verbosity.	0

	refit refit: bool, default=True If set to True, the scores are averaged across all folds, and the coefs and the C that corresponds to the best score is taken, and a final refit is done using these parameters. Otherwise the coefs, intercepts and C that correspond to the best scores across folds are averaged.	True
	intercept_scaling intercept_scaling: float, default=1 Useful only when the solver `liblinear` is used and `self.fit_intercept` is set to `True`. In this case, `x` becomes <code>[x, self.intercept_scaling]</code>, i.e. a "synthetic" feature with constant value equal to `intercept_scaling` is appended to the instance vector. The intercept becomes <code>intercept_scaling * synthetic_feature_weight`</code>. .. note:: The synthetic feature weight is subject to L1 or L2 regularization as all other features. To lessen the effect of regularization on synthetic feature weight	1.0

	(and therefore on the intercept) <code>`intercept_scaling`</code> has to be increased.	
	random_state random_state: int, RandomState instance, default=None Used when <code>`solver='sag'`</code>, <code>`saga`</code> or <code>`liblinear`</code> to shuffle the data. Note that this only applies to the solver and not the cross-validation generator. See :term:`Glossary` for details.	42
	use_legacy_attributes use_legacy_attributes: bool, default=True If True, use legacy values for attributes: - <code>`C_`</code> is an ndarray of shape (n_classes,) with the same value repeated - <code>`l1_ratio_`</code> is an ndarray of shape (n_classes,) with the same value repeated - <code>`coefs_paths_`</code> is a dict with class labels as keys and ndarrays as values - <code>`scores_`</code> is a dict with class labels as keys and ndarrays as values - <code>`n_iter_`</code> is an ndarray of shape (1, n_folds, n_cs) or similar If False, use new values for attributes:	True

	- <code>`C_`</code> is a float - <code>`l1_ratio_`</code> is a float - <code>`coefs_paths_`</code> is an ndarray of shape (n_folds, n_l1_ratios, n_cs, n_classes, n_features) For binary problems (n_classes=2), the 2nd last dimension is 1. - <code>`scores_`</code> is an ndarray of shape (n_folds, n_l1_ratios, n_cs) - <code>`n_iter_`</code> is an ndarray of shape (n_folds, n_l1_ratios, n_cs) .. versionchanged:: 1.10 The default will change from True to False in version 1.10. .. deprecated:: 1.10 <code>`use_legacy_attributes`</code> will be deprecated in version 1.10 and be removed in 1.12.	
--	---	--

Analysis goes here:

Compare Models

```
def find_best_cutoff(y_true, y_probs):
    cutoffs = np.linspace(0.1, 0.9, 81)
    best_cutoff = 0.5
    best_acc = 0
    for c in cutoffs:
        acc = accuracy_score(y_true, (y_probs >= c).astype(int))
        if acc > best_acc:
            best_acc = acc
            best_cutoff = c
    return best_cutoff

models = {'Ridge': ridge_cv, 'LASSO': lasso_cv, 'Elastic Net': elastic_cv}
results = {}
```

```

for name, model in models.items():
    # Get predicted probabilities
    train_probs = model.predict_proba(X_train_scaled)[: , 1]
    test_probs = model.predict_proba(X_test_scaled)[: , 1]

    # Tune cutoff on train data
    best_c = find_best_cutoff(y_train, train_probs)

    # Calculate accuracy on test data using the tuned cutoff
    test_preds = (test_probs >= best_c).astype(int)
    test_acc = accuracy_score(y_test, test_preds)

    results[name] = {'Tuned Cutoff': best_c, 'Test Accuracy': test_acc}

# Display Comparison Table
print(pd.DataFrame(results).T)

```

	Tuned Cutoff	Test Accuracy
Ridge	0.46	0.882086
LASSO	0.47	0.879819
Elastic Net	0.46	0.882086

Analysis goes here:

Bootstrap Confidence Intervals

```

from sklearn.utils import resample
from sklearn.linear_model import LogisticRegression

# Assuming 'LASSO' or 'Elastic Net' performed best; replace `best_model_class`
accordingly
n_iterations = 1000
boot_coefs = []

# Filter to your selected optimal model parameter setting for bootstrapping
best_C = lasso_cv.C_[0]

for i in range(n_iterations):
    # Resample training data
    X_boot, y_boot = resample(X_train_scaled, y_train, random_state=i)

    # Refit model with the fixed optimal hyperparameter
    boot_model = LogisticRegression(l1_ratio=0, solver='saga', C=best_C,

```

```

max_iter=5000)
    boot_model.fit(X_boot, y_boot)
    boot_coefs.append(boot_model.coef_.flatten())

boot_coefs = np.array(boot_coefs)

# Calculate 95% Percentile Confidence Intervals
lower_bounds = np.percentile(boot_coefs, 2.5, axis=0)
upper_bounds = np.percentile(boot_coefs, 97.5, axis=0)

ci_df = pd.DataFrame({
    'Feature': X_train_clean.columns,
    'Optimal_Coef': lasso_cv.coef_.flatten(),
    '2.5%': lower_bounds,
    '97.5%': upper_bounds
})
print(ci_df)

```

	Feature	Optimal_Coef	2.5%	97.5%
0	Age	-0.388041	-0.786700	-0.105255
1	BusinessTravel_NonTravel	-0.233763	-0.533172	-0.153702
2	BusinessTravel_Travel_Frequently	0.309103	0.189153	0.467745
3	BusinessTravel_Travel_Rarely	0.000000	-0.173011	0.089638
4	DistanceFromHome	0.297332	0.147851	0.537364
5	EducationField_HumanResources	0.161330	-0.043989	0.379061
6	EducationField_LifeSciences	0.000000	-0.167275	0.119314
7	EducationField_Marketing	0.076997	-0.123789	0.262534
8	EducationField_Medical	-0.039492	-0.243278	0.083842
9	EducationField_Other	-0.095026	-0.482573	0.069320
10	EducationField_TechnicalDegree	0.105928	-0.052046	0.282768
11	Education	0.000000	-0.211284	0.240075
12	EnvironmentSatisfaction	-0.405029	-0.689840	-0.240178
13	Gender_Female	-0.118815	-0.386127	0.047872
14	HourlyRate	-0.055597	-0.308256	0.144807
15	JobInvolvement	-0.310798	-0.567449	-0.146602
16	JobLevel	0.000000	-0.330696	0.618151
17	JobRole_HealthcareRepresentative	-0.150834	-0.609876	-0.043422
18	JobRole_HumanResources	0.103191	-0.147358	0.293792
19	JobRole_LaboratoryTechnician	0.375714	0.188237	0.566100
20	JobRole_Manager	0.000000	-0.503800	0.184903
21	JobRole_ManufacturingDirector	-0.055522	-0.490713	0.113397
22	JobRole_ResearchDirector	-0.246882	-0.797036	-0.098647

23	JobRole_ResearchScientist	-0.043991	-0.360314	0.088195
24	JobRole_SalesExecutive	0.251224	-0.010114	0.391448
25	JobRole_SalesRepresentative	0.323428	0.161755	0.510975
26	JobSatisfaction	-0.382128	-0.659380	-0.229559
27	MaritalStatus_Divorced	-0.074283	-0.406779	-0.102398
28	MaritalStatus_Married	0.000000	-0.303762	-0.018797
29	MaritalStatus_Single	0.490853	0.260267	0.546107
30	MonthlyRate	0.066643	-0.097159	0.312268
31	NumCompaniesWorked	0.418054	0.285842	0.715534
32	Overtime_Yes	0.780112	0.657749	1.086948
33	PercentSalaryHike	-0.038355	-0.290005	0.136627
34	RelationshipSatisfaction	-0.300524	-0.570694	-0.157501
35	TotalWorkingYears	-0.317243	-0.896572	0.063865
36	TrainingTimesLastYear	-0.162585	-0.430940	0.014687
37	WorkLifeBalance	-0.244077	-0.504903	-0.054342
38	YearsAtCompany	0.000000	-0.439865	0.474686
39	YearsInCurrentRole	-0.382405	-0.810172	-0.132811
40	YearsSinceLastPromotion	0.528322	0.312147	0.945525
41	YearsWithCurrManager	-0.240745	-0.700321	0.138104

Analysis goes here:

Model Comparison using Bayesian Logistic Regression

```
# import bambi as bmb
# import arviz as az

# # Initialize a Bayesian model with a regularizing Normal prior (Ridge
# equivalent)
# bayes_model = bmb.Model("Left ~ Age + DistanceFromHome + C(Overtime)",
# data=train_df, family="bernoulli")
# # Fit using MCMC sampling
# results_bayes = bayes_model.fit(draws=1000, tune=1000, random_seed=42)

# print(az.summary(results_bayes))

# az.plot_trace(results_bayes)
```

Analysis goes here:

Business Implications

The model suggests:

1. Vehicle age is the strongest continuous driver of depreciation.
2. Mileage has a significant but smaller effect.

3. Vehicle model contributes substantial value differences.
4. Trucks generally retain value better than passenger sedans.

Recommendations

Organizations purchasing used vehicles should:

- Prioritize lower-age vehicles when possible.
- Consider mileage alongside age rather than in isolation.
- Use model-specific adjustments when evaluating value.
- Compare observed asking prices against model predictions to identify unusually favorable purchasing opportunities.

Future Work

Future improvements could include:

1. Incorporating original MSRP.
2. Including trim-level information.
3. Adding accident-history variables.
4. Incorporating local economic indicators.

Conclusion

The final regression model explains approximately 82% of the variation in used vehicle selling prices and provides a practical framework for vehicle valuation. Vehicle age, mileage, model, and geographic location all contribute significantly to price determination. The resulting model can support data-driven purchasing and pricing decisions while highlighting areas for future model enhancement.